

신입생 트레이닝
프로그래밍 실습2:
NVMeVirt 활용 SLC cache 구현

담당 조교: 김귀인
(gikim@davinci.snu.ac.kr)

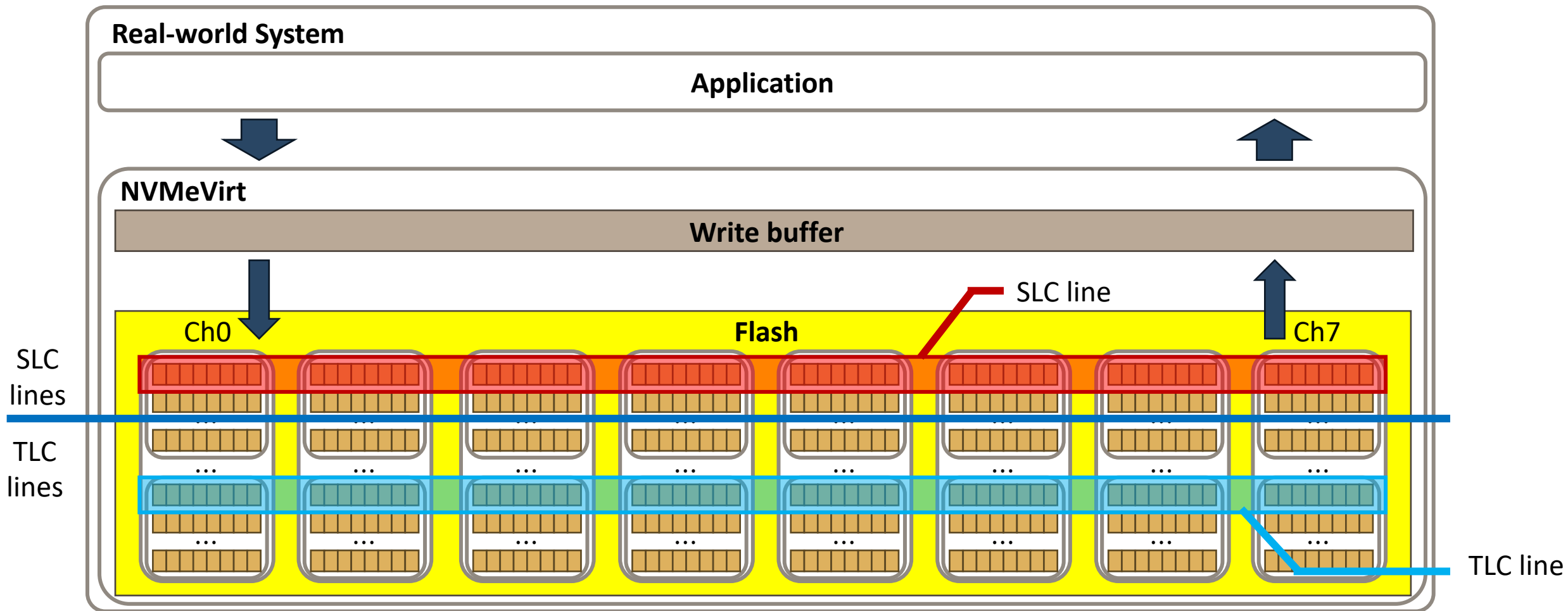
SNU Computer ARchitecture & Embedded Systems Lab

실습목표

- NVMeVirt 에 SLC cache 구현 및
SLC to TLC migration 시 victim selection 별 성능 평가
- SLC caching 참고자료
 - <https://kr.transcend-info.com/embedded/technology/slc-mode>
 - <https://www.atpinc.com/blog/what-is-SLC-cache-difference-between-Dynamic-Static-SLC-cache>
- 실습 내용
 - SLC 구현

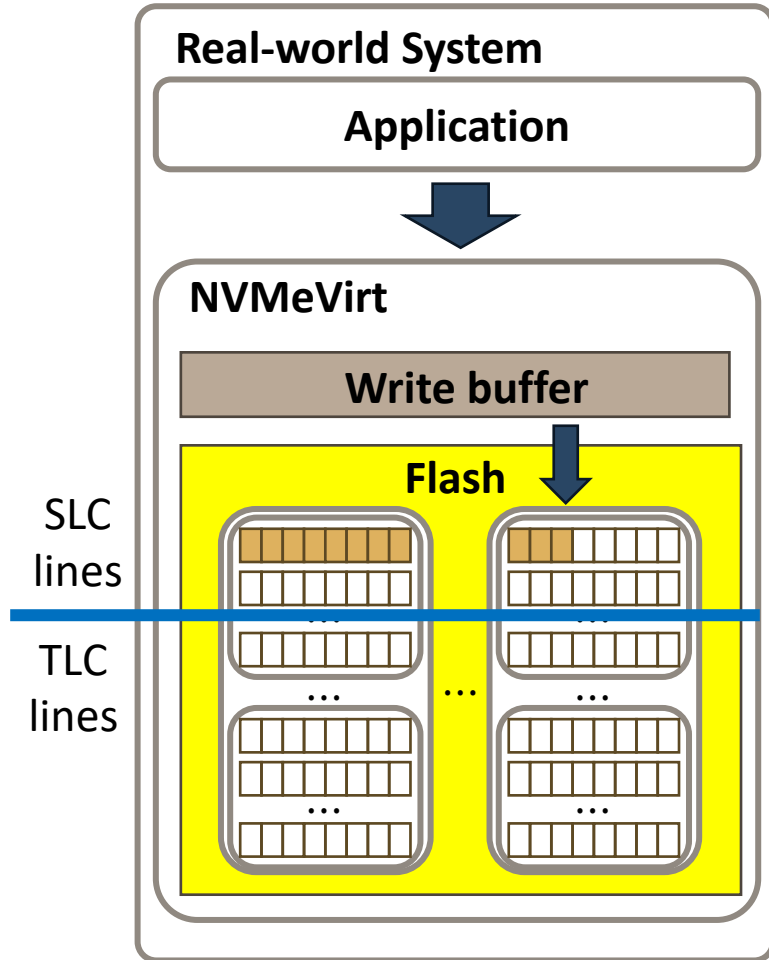
구조 개요

- NVMeVirt with SLC cache

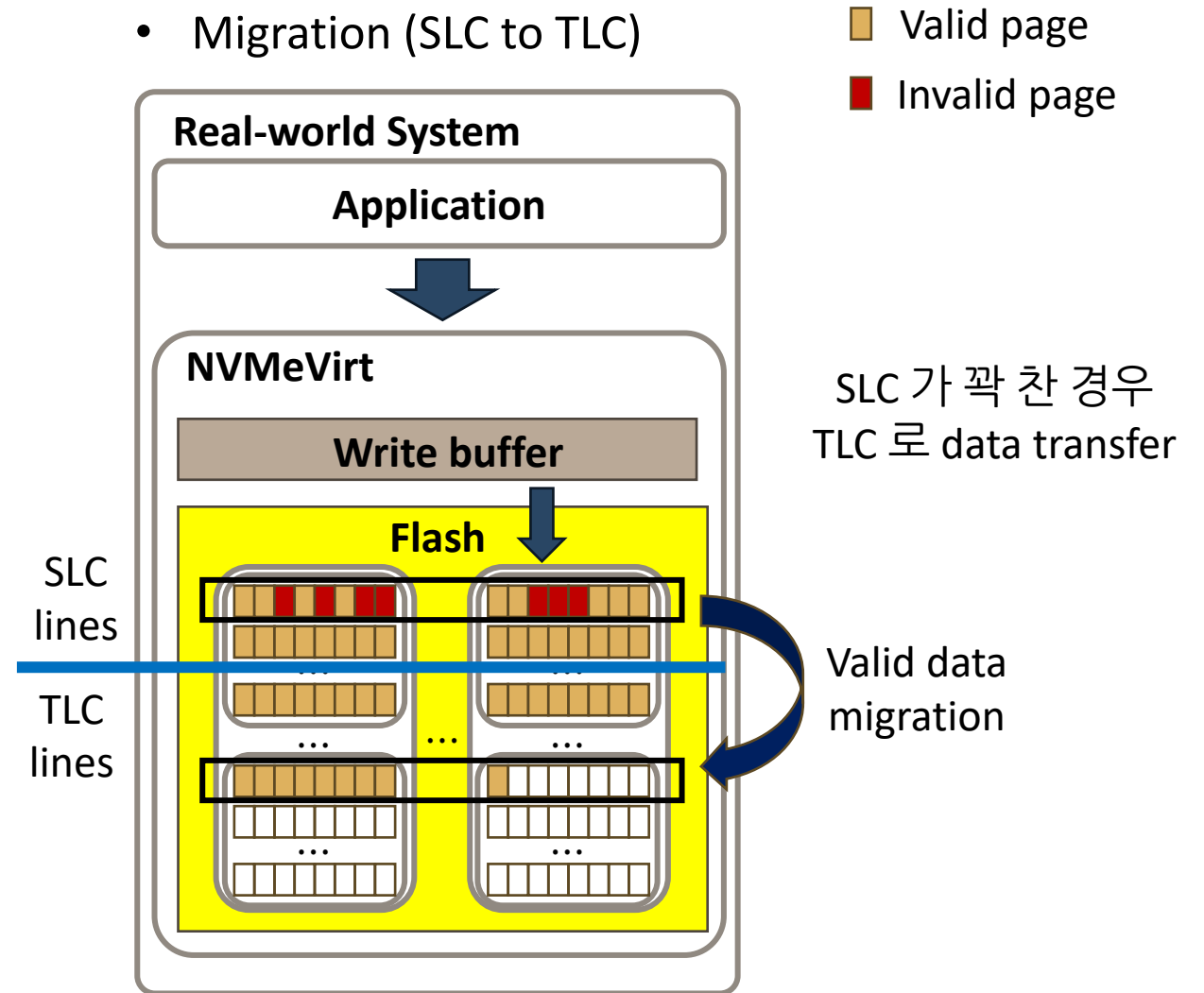


동작 개요

- Write (in SLC)

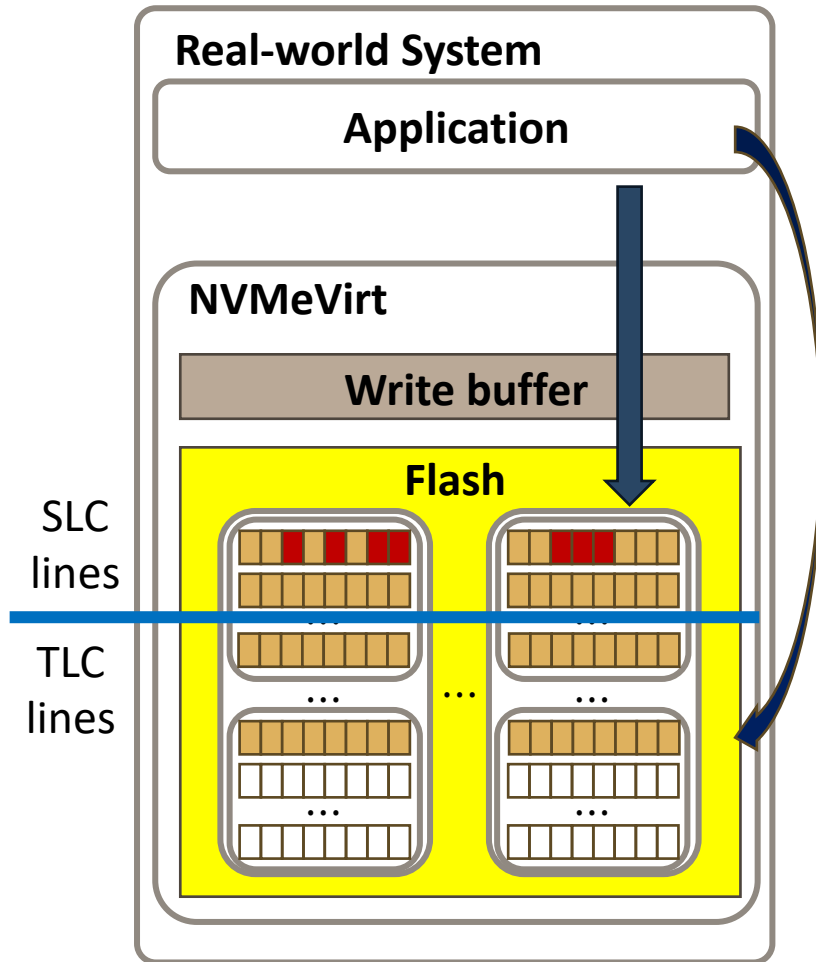


- Migration (SLC to TLC)

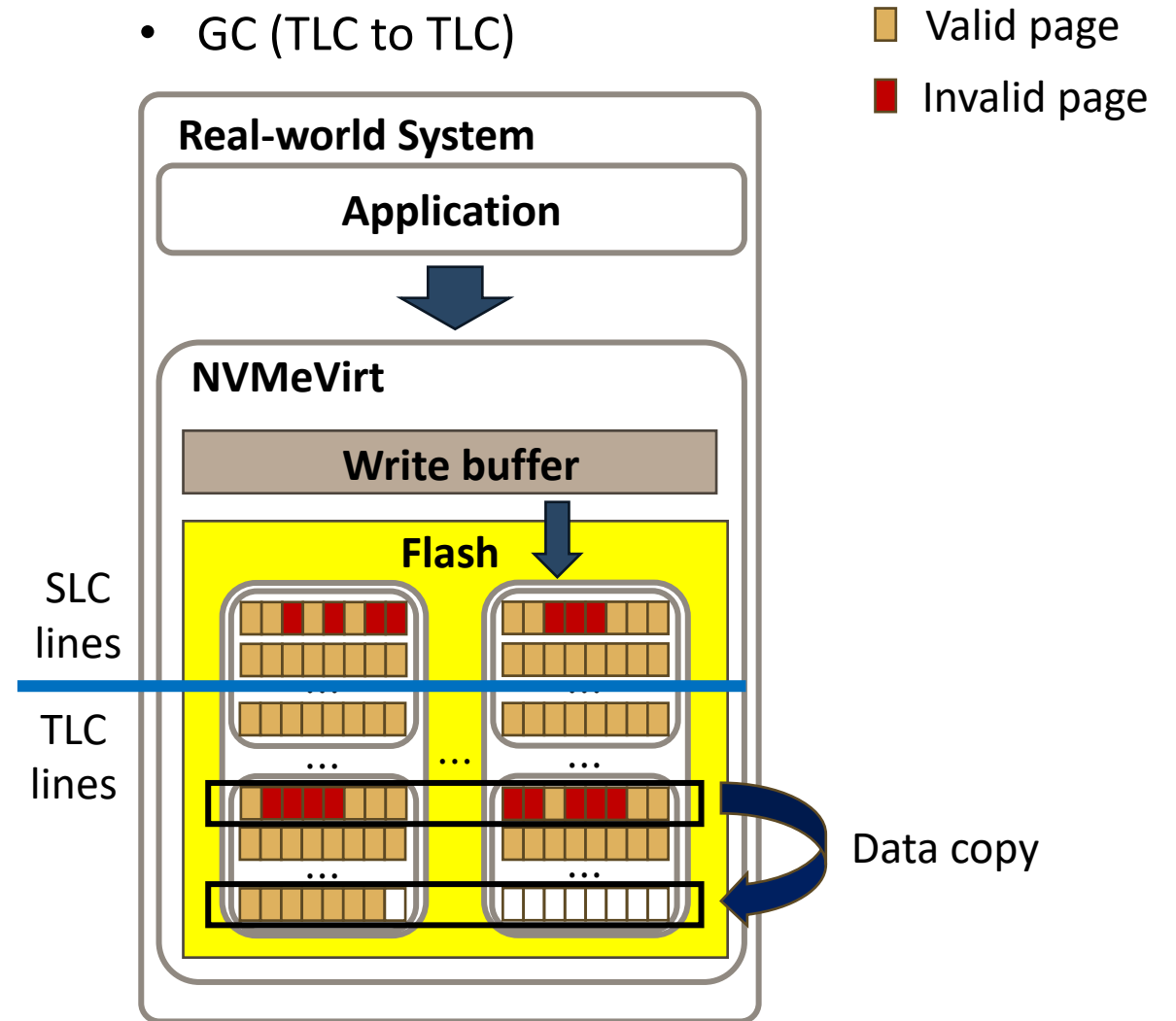


동작 개요 (cont.)

- Read (SLC or TLC)



- GC (TLC to TLC)



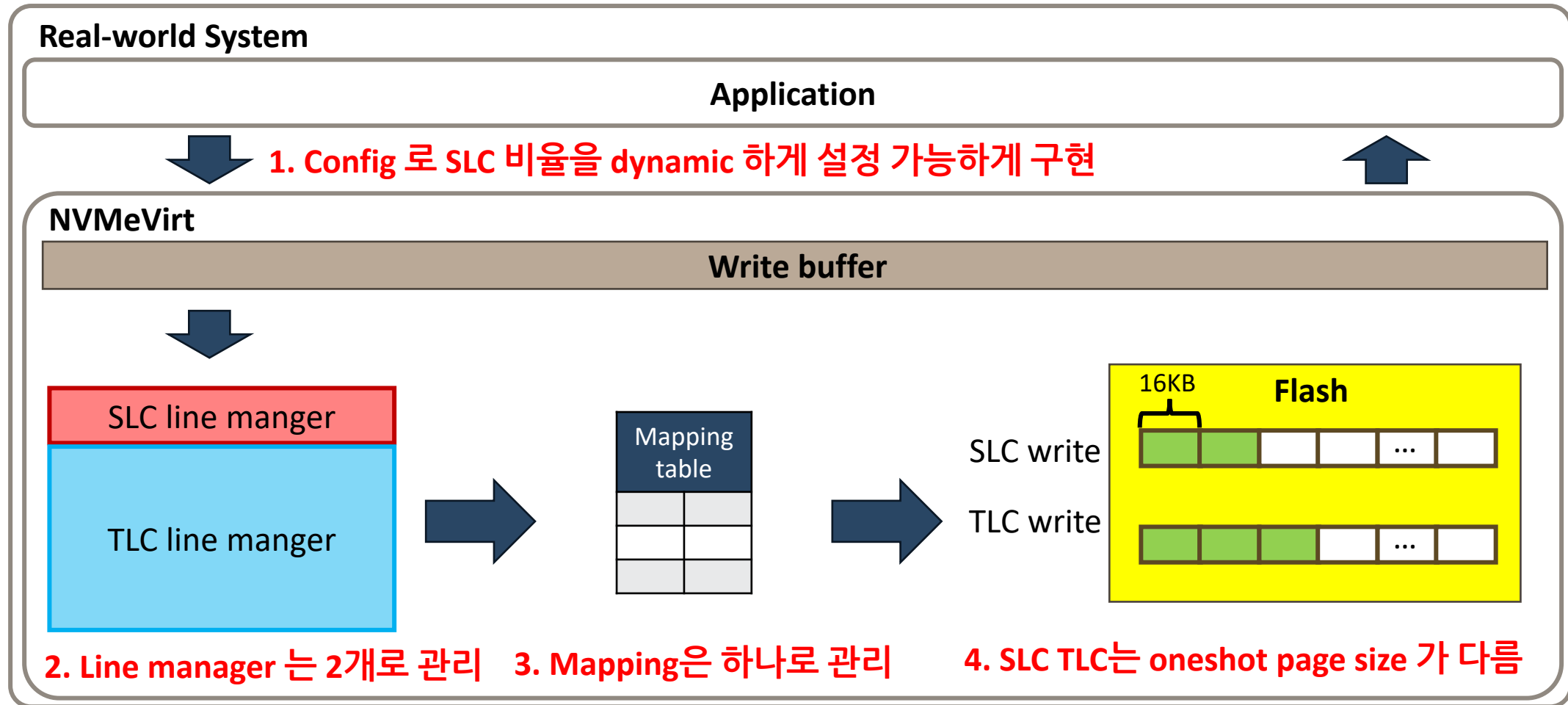
실습 1: SLC cache 구현

- NVMeVirt에 제공되어 있는 Conventional FTL에 SLC cache 를 구현 및 비교
- 평가 방법
 - 정상작동 여부 확인: SLC 크기를 고려하여 벤치마크를 수행하여 다음 항목들을 확인
 - SLC 에만 read/write 가 수행되는 지
 - SLC 가 차서 TLC에 read/write 가 수행되는 지
 - 기존 NVMeVirt와 성능 비교 평가
 - 벤치마크 프로그램: Filebench, FIO 등
 - 평가 요소: Throughput, Latency

실습 2: Migration victim selection 구현 및 비교

- Greedy(baseline), random, FIFO, Cost-Benefit policy를 각기 구현하여 비교
- 평가 방법
 - 각 policy마다 아래 요소 측정하여 비교
 - 벤치마크 프로그램: Filebench, FIO 등
 - 평가 요소: Throughput, Latency
- 결과물 제출 (8월 28일까지 gikim@davinci.snu.ac.kr로 제출)
 - 구현한 코드 (새롭게 구현한 파일만)
 - 결과 보고서 (측정 결과 그래프, 결과에 대한 간단한 분석)

전체 System 구현 유의사항



- Skeleton code로 제공되는 `ssd_config.h` 설정을 사용할 것